



Security Assessment Report

XAUm

January 20, 2026

Summary

The Sec3 team was engaged to conduct a thorough security analysis of the XAUm.

The artifact of the audit was the source code of the following programs, excluding tests, in a private repository.

The initial audit focused on the following versions and revealed 3 issues or questions.

#	Task	Type	Commit
P1	XAUm Minter	Solana	41c7d970469ed63518ae943b878a8a873ff0f7e2
P2	XAUm Token	Solana	41c7d970469ed63518ae943b878a8a873ff0f7e2

This report provides a detailed description of the findings and their respective resolutions.

Table of Contents

Result Overview

Findings in Detail

[P1-I-01] Ineffective timestamp validation

[P1-Q-01] Missing on-chain validation for for_token in mint/redeem requests

[P2-L-01] Minimum delay policy can be bypassed during Token creation

Appendix: Methodology and Scope of Work

3

4

4

5

7

8

Result Overview

Issue	Impact	Status
XAUM MINTER		
[P1-I-01] Ineffective timestamp validation	Info	Acknowledged
[P1-Q-01] Missing on-chain validation for for_token in mint/redeem requests	Question	Resolved
XAUM TOKEN		
[P2-L-01] Minimum delay policy can be bypassed during Token creation	Low	Resolved

Findings in Detail

XAUM MINTER

[P1-I-01] Ineffective timestamp validation

The `xaum-minter` program validates mint and redeem requests using a client-provided `timestamp`. The intention is to ensure requests are processed within a specific window (`DELAY_MAX`) from their creation.

```
/* xaum-solana/programs/xaum-minter/src/lib.rs */
125 | let clock = Clock::get()?;
126 | require!(
127 |     clock.unix_timestamp <= timestamp + DELAY_MAX,
128 |     ErrorCode::InvalidTimestamp
129 | );
```

The validation only checks that the current time has not passed the `timestamp + DELAY_MAX` deadline. Crucially, it does not verify that the `timestamp` is not in the future.

A malicious or buggy client can submit a request with a `timestamp` set far in the future (e.g., `now + 1 year`). The condition `now <= future_time + DELAY_MAX` will evaluate to true. This allows the request to remain valid for an indefinite period, effectively bypassing the expiry mechanism and potentially disrupting off-chain order processing systems that rely on this timestamp.

Resolution

The team clarified that the `timestamp` parameter is intended to prompt users to submit the mint request promptly after receiving a quote price. It is mainly used by our frontend and does not require strict on-chain enforcement.

XAUM MINTER

[P1-Q-01] Missing on-chain validation for for_token in mint/redeem requests

In the `xaum-minter` program, the `request_to_mint` and `request_to_redeem` instructions strictly validate the `transferred_token` (the token being sent by the user) against the `accepted_by_a` and `accepted_by_b` whitelists, respectively.

However, the `for_token` (the token the user expects to receive in return) is **not validated** against any on-chain list.

```
/* xaum-solana/programs/xaum-minter/src/lib.rs */
101 | // preprice and slippage are client-provided hints, not validated on-chain
102 | pub fn request_to_mint(
103 |     ctx: Context<RequestMint>,
104 |     transferred_token: Pubkey,
105 |     for_token: Pubkey,
106 |     amount: u64,
107 |     preprice: u64,
108 |     slippage: u64,
109 |     timestamp: i64,
110 | ) -> Result<()> {
111 |     let state = &ctx.accounts.state;
112 |     require!(
113 |         for_token == ctx.accounts.for_token.key(),
114 |         ErrorCode::InvalidForToken
115 |     );

/* xaum-solana/programs/xaum-minter/src/lib.rs */
313 | #[derive(Accounts)]
314 | pub struct RequestMint<'info> {
...
338 |     /// CHECK: token mint of for token
339 |     #[account(
340 |         mint::token_program = token_program_for_token
341 |     )]
342 |     pub for_token: Box<InterfaceAccount<'info, Mint>>,

/* xaum-solana/programs/xaum-minter/src/lib.rs */
160 | pub fn request_to_redeem(
161 |     ctx: Context<RequestRedeem>,
162 |     transferred_token: Pubkey,
163 |     for_token: Pubkey,
164 |     amount: u64,
165 |     preprice: u64,
166 |     slippage: u64,
167 |     timestamp: i64,
168 | ) -> Result<()> {
```

A user can call `request_to_mint` transferring a valid accepted token (e.g., USDC) but requesting an arbitrary or invalid token (e.g., a random shitcoin or the same USDC back) as the `for_token`.

While the actual minting/transfer happens off-chain via the event processor, this lack of on-chain validation delegates all security checks to the off-chain component. If the off-chain parser assumes that

"successful transaction = valid request," it could lead to incorrect state processing or failed settlement attempts.

Is it intended to leave `for_token` completely open-ended to support future flexibility, or should it be validated against a reciprocal whitelist?

Resolution

This issue has been fixed by `16f79c2`.

XAUM TOKEN

[P2-L-01] Minimum delay policy can be bypassed during Token creation

The `xaum-token` program defines `MIN_ACCEPTABLE_DELAY` (3600 seconds) to ensure a minimum timelock for sensitive operations like minting or ownership transfer. This limit is correctly enforced in the `set_delay` instruction.

However, the `create_token` instruction initializes the state with a user-provided `delay` without performing any validation against this minimum constant.

```
/* xaum-solana/programs/xaum-token/src/instructions/authority.rs */
177 | pub fn set_delay(ctx: Context<OwnerOp>, new_delay: i64) -> Result<()> {
178 |     require!(
179 |         new_delay >= MIN_ACCEPTABLE_DELAY,
180 |         ErrorCode::DelayBelowMinimum
181 |     );
182 |     require!(
183 |         new_delay <= MAX_ACCEPTABLE_DELAY,
184 |         ErrorCode::DelayExceedsMaximum
185 |     );

/* xaum-solana/programs/xaum-token/src/instructions/create.rs */
047 | pub fn create_token(
048 |     ctx: Context<CreateToken>,
049 |     token_name: String,
050 |     token_symbol: String,
051 |     token_uri: String,
052 |     delay: i64,
053 | ) -> Result<()> {
054 |     // Initialize state
055 |     let state = &mut ctx.accounts.state;
056 |     let bump = ctx.bumps.state;
057 |     let payer = *ctx.accounts.payer.key;
058 |     state.init(payer, delay, bump);
```

This allows the deployer to initialize the token with `delay = 0`, bypassing the protocol's security policy. While the owner can inadvertently fix this by calling `set_delay` later (which will then enforce the minimum), the initial state remains non-compliant and allows instant operations.

Resolution

The team addressed this issue by adding validation checks to the `create_token` instruction in commit `cb9a3b1`. The implementation now ensures that the initial `delay` is non-negative and does not exceed the `MAX_ACCEPTABLE_DELAY`. The team opted to allow values below the `MIN_ACCEPTABLE_DELAY` during initialization to provide flexibility for the initial protocol setup.

Appendix: Methodology and Scope of Work

Assisted by the Sec3 Scanner developed in-house, the manual audit particularly focused on the following work items:

- Check common security issues.
- Check program logic implementation against available design specifications.
- Check poor coding practices and unsafe behavior.
- The soundness of the economics design and algorithm is out of scope of this work

DISCLAIMER

The instance report ("Report") was prepared pursuant to an agreement between Coderect Inc. d/b/a Sec3 (the "Company") and Flying Otter Limited dba Matrixdock XAUm (the "Client"). This Report solely includes the results of a technical assessment of a specific build and/or version of the Client's code specified in the Report ("Assessed Code") by the Company. The sole purpose of the Report is to provide the Client with the results of the technical assessment of the Assessed Code. The Report does not apply to any other version and/or build of the Assessed Code. Regardless of the contents of the Report, the Report does not (and should not be interpreted to) provide any warranty, representation or covenant that the Assessed Code: (i) is error and/or bug free, (ii) has no security vulnerabilities, and/or (iii) does not infringe any third-party rights. Moreover, the Report is not, and should not be considered, an endorsement by the Company of the Assessed Code and/or of the Client. Finally, the Report should not be considered investment advice or a recommendation to invest in the Assessed Code and/or the Client.

This Report is considered null and void if the Report (or any portion thereof) is altered in any manner.

ABOUT

The Sec3 audit team comprises a group of computer science professors, researchers, and industry veterans with extensive experience in smart contract security, program analysis, testing, and formal verification. We are also building automated security tools that incorporate static analysis, penetration testing, and formal verification.

At Sec3, we identify and eliminate security vulnerabilities through the most rigorous process and aided by the most advanced analysis tools.

For more information, check out our [website](#) and follow us on [twitter](#).

